来源文件：docs/week-03/测试报告.md（测试报告）

V1.0 测试报告

日期：2026-10-16。分支：`chore/week3/v1-release`。测试库：`.env` 中的 `MYSQL_TEST_DATABASE`（`syncflow_test`），不写开发库 `syncflow`。

测试环境

系统环境	版本
Python	3.13.2
MySQL	8.0.46
Redis	7.4.11
操作系统	Windows11Pro 26H2
pytest	9.1.1
Docker	29.8.1
Docker Compose	v5.5.1

执行命令

在仓库的 `backend` 目录：

```
.\.venv\Scripts\python.exe -m pytest -q --tb=line
```



解释器是 `backend\.venv` 里的 Python 3.13。

结果

`65 passed in 8.06s`。没有失败，没有跳过。

用例清单

下面 65 条与本次 `pytest` 收集到的用例一一对应，每条都写出 `文件::函数`。

创建参数、数据库异常、10MB 边界

用例	覆盖
`test_jobs_api.py::test_create_job_success`	201、只返回 `id/name/status/created_at`、文件落盘、入测试队列、不写开发库
`test_jobs_api.py::test_create_rejects_invalid_parameters`	缺文件 `INVALID_REQUEST`、非 csv `INVALID_FILE_EXTENSION`、名称超长、非法状态
`test_jobs_api.py::test_upload_accepts_exact_ten_megabytes_and_rejects_one_extra_byte`	正好 10MB 可以创建，多 1 字节 `FILE_TOO_LARGE`，超限文件不留在磁盘
`test_jobs_api.py::test_create_rejects_oversize_file`	把上限降到 0 后，任意内容都是 `FILE_TOO_LARGE`
`test_jobs_api.py::test_database_error_is_hidden`	数据库异常返回 500 `INTERNAL_ERROR`，响应不含连接串、密码和堆栈，并删掉已写文件

分页边界

用例	覆盖
`test_jobs_api.py::test_list_pagination_boundaries`	任务列表排序、超出最后一页、默认 20、最大 100、`page=0` 与 `page_size=101` 为 400、`SUCCESS / PARTIAL_SUCCESS` 筛选、详情与 404

用例	覆盖
test_jobs_api.py::test_openapi_lists_job_routes	OpenAPI 含创建、列表、详情
test_errors_api.py::test_error_list_paginates_in_row_then_id_order	错误明细按 row_number，id 排序，空行号在前，超出最后一页
test_errors_api.py::test_error_list_rejects_bad_pages_and_missing_or_empty_jobs	无错误、404、page=0、page_size=101
test_errors_api.py::test_openapi_lists_error_route	OpenAPI 含 GET /api/v1/jobs/{job_id}/errors

CSV 读取 test_csv_reader.py

用例	覆盖
test_csv_reader.py::test_utf8_keeps_text_and_skips_blank_lines	UTF-8 保留中文；空行不计入记录，行号仍按文件原行
test_csv_reader.py::test_utf8_bom_does_not_prefix_header	UTF-8 BOM 不留在表头第一个字段上
test_csv_reader.py::test_crlf_and_quoted_newline	CRLF 可解析；引号内换行算同一条记录，起始行号为 3
test_csv_reader.py::test_comma_only_row_is_kept	只有逗号的行是空字段，保留给行级校验，不算空行
test_csv_reader.py::test_empty_file_is_file_empty	零字节文件为 FILE_EMPTY
test_csv_reader.py::test_bom_only_is_file_empty	只有 BOM 为 FILE_EMPTY
test_csv_reader.py::test_header_only_is_file_empty	只有表头、后面是空行为 FILE_EMPTY
test_csv_reader.py::test_missing_file_is_unreadable	路径不存在为 FILE_UNREADABLE
test_csv_reader.py::test_directory_is_unreadable	路径是目录为 FILE_UNREADABLE
test_csv_reader.py::test_unclosed_quote_is_unreadable	引号不闭合为 FILE_UNREADABLE
test_csv_reader.py::test_invalid_utf8_is_encoding_invalid	非法 UTF-8 字节为 FILE_ENCODING_INVALID
test_csv_reader.py::test_gbk_text_is_encoding_invalid	GBK 中文为 FILE_ENCODING_INVALID

CSV 解析 test_csv_parser.py

用例	覆盖
test_csv_parser.py::test_file_unreadable	缺失文件和读失败都是 FILE_UNREADABLE，不再做行级校验
test_csv_parser.py::test_file_encoding_invalid	非 UTF-8 为 FILE_ENCODING_INVALID
test_csv_parser.py::test_file_empty	空文件、只有表头、表头不对但没有数据行，都是 FILE_EMPTY
test_csv_parser.py::test_csv_header_invalid_missing_duplicate_order_and_extra	缺列、重复列、顺序错、多余列、大小写不符、表头带空格，都是 CSV_HEADER_INVALID
test_csv_parser.py::test_csv_header_invalid_wins_over_row_limit	表头错误优先于行数上限
test_csv_parser.py::test_read_csv_still_returns_table_for_bad_header	坏表头时 read_csv 仍返回 CsvTable，表头判断不在读取这一步
test_csv_parser.py::test_file_too_many_rows	10000 行通过，10001 行为 FILE_TOO_MANY_ROWS
test_csv_parser.py::test_file_too_many_rows_uses_settings	不传上限时使用 MAX_RECORDS_PER_JOB
test_csv_parser.py::test_blank_lines_do_not_count_toward_limit	空行不计入 10000 行上限
test_csv_parser.py::test_row_column_count_mismatch	列数不对为 ROW_COLUMN_COUNT_MISMATCH
test_csv_parser.py::test_field_required	四个必填字段为空都是 FIELD_REQUIRED，并带上实际列名
test_csv_parser.py::test_external_id_invalid	空格、非法字符、超长为 EXTERNAL_ID_INVALID
test_csv_parser.py::test_external_id_duplicate_is_case_insensitive	同一任务内 external_id 大小写不敏感地判重
test_csv_parser.py::test_duplicate_id_is_taken_when_other_fields_fail	同行还有别的错误时，重复的 external_id 仍然占住
test_csv_parser.py::test_name_too_long	名称超过 128 为 NAME_TOO_LONG
test_csv_parser.py::test_amount_invalid	负数、超过两位小数、科学计数法、超出 DECIMAL(12,2) 为 AMOUNT_INVALID
test_csv_parser.py::test_amount_normalizes_without_float	19、19.9、19.90 用 Decimal 收成两位小数，不经过 float
test_csv_parser.py::test_date_invalid	斜线、带时间、不存在的日期为 DATE_INVALID
test_csv_parser.py::test_valid_row_keeps_trimmed_name_and_raw_summary	合法行去掉名称两端空格；错误行的 raw_row 保留原始值
test_csv_parser.py::test_one_row_reports_fields_in_header_order	同一行按表头顺序报 EXTERNAL_ID_INVALID、FIELD_REQUIRED、AMOUNT_INVALID、DATE_INVALID

导入 test_import_service.py

用例	覆盖
test_import_service.py::test_success_import_writes_records_and_terminal_stats	两行都成功：SUCCESS 2/2/0，external_id 大写，金额为两位小数，不写开发库
test_import_service.py::test_partial_success_keeps_good_rows_and_row_errors	一行成功、一行金额非法：PARTIAL_SUCCESS，2/1/1，错误码 AMOUNT_INVALID
test_import_service.py::test_all_rows_invalid_finishes_failed	全部行级失败：FAILED，2/0/2，不写 sync_records
test_import_service.py::test_header_invalid_is_file_level_failure	表头错误：FAILED，0/0/0，一条 CSV_HEADER_INVALID row_number 为空
test_import_service.py::test_empty_file_is_file_level_failure	只有表头：FAILED，0/0/0，一条 FILE_EMPTY
test_import_service.py::test_ten_thousand_rows_can_be_written	正好 10000 行写入成功，没有错误行
test_import_service.py::test_failed_batch_does_not_roll_back_other_batches	后一批唯一约束冲突不回滚前一批；消息是「第 2 批写入失败（唯一约束冲突）」，码为 INTERNAL_ERROR
test_import_service.py::test_multiple_batch_failures_summarize_batch_numbers	多批失败汇总成「第 1 2 批写入失败，共 N 批」
test_import_service.py::test_repeat_import_does_not_write_again	已经结束的任务再次导入会跳过，不重复写记录
test_import_service.py::test_batch_log_separates_csv_line_from_data_row_progress	批日志用 csv_line 和 data_rows，不再使用 row=已处理/总数
test_import_service.py::test_record_repository_keeps_earlier_batches_when_a_later_batch_conflicts	仓库层只回滚冲突的那一批，更早的批次仍在

样例、错误码与配置

用例	覆盖
test_samples.py::test_release_samples_match_expected_stats	ok.csv、bom.csv、row-errors.csv、bad-header.csv、empty.csv 的终态和计数与 samples/README.md 一致
test_samples.py::test_import_rejects_non_utf8_and_too_many_rows	GBK 为 FILE_ENCODING_INVALID；10001 行为 FILE_TOO_MANY_ROWS，计数 0/0/0，不写 sync_records
test_sync_foundations.py::test_max_records_per_job_reads_env	MAX_RECORDS_PER_JOB 默认 10000，并读取环境变量
test_sync_foundations.py::test_sync_error_codes_match_week3_spec	文件级 5 码、行级 7 码，以及每个码的 field_name
test_sync_foundations.py::test_cleanup_removes_records_and_errors	测试清理会先删 sync_records 和 sync_errors

Worker test_worker.py

用例	覆盖
test_worker.py::test_job_timeout_seconds_reads_env	JOB_TIMEOUT_SECONDS 读取环境变量，用例里设为 45
test_worker.py::test_worker_reaches_success_partial_and_failed	同一 Worker 路径得到 SUCCESS、PARTIAL_SUCCESS、FAILED
test_worker.py::test_worker_marks_missing_file_unreadable	文件被删掉后为 FILE_UNREADABLE，计数 0
test_worker.py::test_worker_unexpected_error_does_not_stay_running	未捕获异常变成 FAILED，消息是「Worker 处理失败」，不含堆栈
test_worker.py::test_worker_second_delivery_does_not_write_again	同一任务第二次投递不再写第二行
test_worker.py::test_mark_failed_does_not_overwrite_terminal_jobs	只把 PENDING / RUNNING 标失败；SUCCESS 的状态、统计和记录保持不变

用例	覆盖
test_worker.py::test_reclaim_stale_running_leaves_fresh_jobs	启动时把超时 RUNNING 收成「任务处理超时」，新的 RUNNING 保持不动

端到端验证

本机宿主端口 8000 落在 Windows 保留区间（netsh interface ipv4 show excludedportrange protocol=tcp 显示 7998–8097），8000:8000 无法绑定，因此用 docker-compose.e2e.yml 只把 API 的宿主端口改为 18000；容器内仍是 8000，web 经 API_PROXY_TARGET=http://api:8000 走容器网络，不受影响。

搭建整条链路

根目录运行：

```
docker compose -f docker-compose.yml -f docker-compose.e2e.yml up -d --build
```

```
docker compose -f docker-compose.yml -f docker-compose.e2e.yml ps
```

```
[+] up 7/7
 ✓ Image syncflow-backend       Built
 ✓ Image syncflow-web           Built
 ✓ Container syncflow-mysql-1   Healthy
 ✓ Container syncflow-redis-1   Healthy
 ✓ Container syncflow-worker-1  Started
 ✓ Container syncflow-api-1     Started
 ✓ Container syncflow-web-1     Started
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv) took 53s
❯ docker compose -f docker-compose.yml -f docker-compose.e2e.yml ps
NAME               IMAGE              COMMAND                  SERVICE   CREATED              STATUS                 PORTS
syncflow-api-1     syncflow-backend   "uvicorn app.main:ap…"   api       About a minute ago   Up About a minute      0.0.0.0:18000->8000/tcp, [::]:18000->8000/tcp
syncflow-mysql-1   mysql:8.0          "docker-entrypoint.s…"   mysql     17 hours ago         Up 6 hours (healthy)   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
syncflow-redis-1   redis:7-alpine     "docker-entrypoint.s…"   redis     2 weeks ago          Up 6 hours (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
syncflow-web-1     syncflow-web       "docker-entrypoint.s…"   web       About a minute ago   Up About a minute      0.0.0.0:5173->5173/tcp, [::]:5173->5173/tcp
syncflow-worker-1  syncflow-backend   "python -m app.worker"   worker    About a minute ago   Up About a minute
```

探活

```
curl.exe -s http://127.0.0.1:18000/healthz
curl.exe -s http://127.0.0.1:18000/readyz
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv)
❯ curl.exe -s http://127.0.0.1:18000/healthz
· curl.exe -s http://127.0.0.1:18000/readyz
{"data":{"status":"ok","service":"api","time":"2026-10-16T07:41:27.140574+00:00"},"meta":{}}{"data":{"status":"ok","service"
:"api","checks":{"mysql":"ok","redis":"ok"}},"meta":{}}
```

readyz 的 data.checks 里 mysql 和 redis 都应是 ok

成功任务

```
$t0 = Get-Date
$ok = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
  -F "name=e2e-ok" -F "file=@samples/ok.csv;type=text/csv" | ConvertFrom-Json
"POST 耗时(ms) = " + [int]((Get-Date) - $t0).TotalMilliseconds
$okId = $ok.data.id
"okId = $okId"
"创建时状态 = " + $ok.data.status          # 期望 PENDING
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv)
❯ $t0 = Get-Date
· $ok = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
·   -F "name=e2e-ok" -F "file=@samples/ok.csv;type=text/csv" | ConvertFrom-Json
· "POST 耗时(ms) = " + [int]((Get-Date) - $t0).TotalMilliseconds
· $okId = $ok.data.id
· "okId = $okId"
· "创建时状态 = " + $ok.data.status          # 期望 PENDING
POST 耗时(ms) = 196
okId = 5862d534-bdf4-4916-a8f2-ca07126bee75
创建时状态 = PENDING
```

终态：

```
Start-Sleep -Seconds 3
(curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$okId" | ConvertFrom-Json).data
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv)
❯ Start-Sleep -Seconds 3
· (curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$okId" | ConvertFrom-Json).data

id                : 5862d534-bdf4-4916-a8f2-ca07126bee75
name              : e2e-ok
status            : SUCCESS
source_file_name  : ok.csv
total_records     : 2
success_records   : 2
failed_records    : 0
retry_count       : 0
last_error_code   :
last_error_message :
created_at        : 10/16/2026 7:42:43 AM
started_at        : 10/16/2026 7:42:43 AM
finished_at       : 10/16/2026 7:42:43 AM
```

失败任务

```
$bad = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
  -F "name=e2e-bad-header" -F "file=@samples/bad-header.csv;type=text/csv" | ConvertFrom-Json
$badId = $bad.data.id
Start-Sleep -Seconds 3
(curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$badId" | ConvertFrom-Json).data
(curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$badId/errors" | ConvertFrom-Json).data
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv)
❯ $bad = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
·   -F "name=e2e-bad-header" -F "file=@samples/bad-header.csv;type=text/csv" | ConvertFrom-Json
· $badId = $bad.data.id
· Start-Sleep -Seconds 3
· (curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$badId" | ConvertFrom-Json).data
· (curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$badId/errors" | ConvertFrom-Json).data

id                : 868232dd-f454-4adc-b5f1-eedfa2348130
name              : e2e-bad-header
status            : FAILED
source_file_name  : bad-header.csv
total_records     : 0
success_records   : 0
failed_records    : 0
retry_count       : 0
last_error_code   : CSV_HEADER_INVALID
last_error_message : 表头缺失、重复、顺序错误或包含额外字段
created_at        : 10/16/2026 7:49:21 AM
started_at        : 10/16/2026 7:49:21 AM
finished_at       : 10/16/2026 7:49:21 AM

job_id        : 868232dd-f454-4adc-b5f1-eedfa2348130
row_number    :
field_name    :
error_code    : CSV_HEADER_INVALID
error_message : 表头缺失、重复、顺序错误或包含额外字段
raw_row       :
created_at    : 10/16/2026 7:49:21 AM
```

期望：FAILED、0 / 0 / 0、last_error_code = CSV_HEADER_INVALID；错误明细一条，row_number 为 null。

Worker

```
docker compose exec -T mysql mysql --user=syncflow --password=syncflow123 --host=127.0.0.1 syncflow -e "SELECT
id,name,status,total_records,success_records,failed_records,last_error_code FROM sync_jobs WHERE id IN
('$okId','$badId'); SELECT job_id,COUNT(*) AS recs FROM sync_records WHERE job_id='$okId' GROUP BY job_id; SELECT
job_id,row_number,error_code FROM sync_errors WHERE job_id='$badId';"

docker compose logs --tail 40 worker
```

```
syncflow on  chore/week3/v1-release [!?] via  v3.13.2 (.venv) took 3s
❯ docker compose exec -T mysql mysql --user=syncflow --password=syncflow123 --host=127.0.0.1 syncflow -e "SELECT id,name,sta
tus,total_records,success_records,failed_records,last_error_code FROM sync_jobs WHERE id IN ('$okId','$badId'); SELECT job_i
d,COUNT(*) AS recs FROM sync_records WHERE job_id='$okId' GROUP BY job_id; SELECT job_id,row_number,error_code FROM sync_err
ors WHERE job_id='$badId';"
·
· docker compose logs --tail 40 worker
mysql: [Warning] Using a password on the command line interface can be insecure.
id      name    status  total_records   success_records failed_records  last_error_code
5862d534-bdf4-4916-a8f2-ca07126bee75    e2e-ok  SUCCESS 2       2       0       NULL
868232dd-f454-4adc-b5f1-eedfa2348130    e2e-bad-header  FAILED  0       0       0       CSV_HEADER_INVALID
job_id  recs
5862d534-bdf4-4916-a8f2-ca07126bee75    2
ERROR 1064 (42000) at line 1: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server v
ersion for the right syntax to use near ',error_code FROM sync_errors WHERE job_id='868232dd-f454-4adc-b5f1-eedfa2348130'' a
t line 1

What's next:
    Debug this Compose error with Gordon → docker ai "help me fix this compose error"
worker-1  | 2026-10-16 07:20:54,964 INFO service=worker worker started
worker-1  | 2026-10-16 07:42:43,921 INFO service=worker job_id=5862d534-bdf4-4916-a8f2-ca07126bee75 rows=2 accepted=2
worker-1  | 2026-10-16 07:42:43,941 INFO service=worker job_id=5862d534-bdf4-4916-a8f2-ca07126bee75 batch=1 csv_line=3 data_
rows=2/2 result=ok elapsed_ms=19
worker-1  | 2026-10-16 07:42:43,949 INFO service=worker job_id=5862d534-bdf4-4916-a8f2-ca07126bee75 status=SUCCESS total=2 s
uccess=2 failed=0 elapsed_ms=53
worker-1  | 2026-10-16 07:46:24,646 INFO service=worker job_id=4e6272de-a8b6-4fe2-8396-73f516fb0ec5 file_error=CSV_HEADER_IN
VALID
worker-1  | 2026-10-16 07:46:24,658 INFO service=worker job_id=4e6272de-a8b6-4fe2-8396-73f516fb0ec5 status=FAILED total=0 su
ccess=0 failed=0 elapsed_ms=30
worker-1  | 2026-10-16 07:49:21,802 INFO service=worker job_id=868232dd-f454-4adc-b5f1-eedfa2348130 file_error=CSV_HEADER_IN
VALID
worker-1  | 2026-10-16 07:49:21,812 INFO service=worker job_id=868232dd-f454-4adc-b5f1-eedfa2348130 status=FAILED total=0 su
ccess=0 failed=0 elapsed_ms=28
```

页面检查

浏览器打开 http://127.0.0.1:5173/jobs/new ，上传sample/ok.csv，看到“已完成”。上传samples/bad-header.csv，看到错误明细

SyncFlow 任务查询

导入-ok-2026-10-16

已完成

返回列表

源文件名

ok.csv

总记录数

2

成功数

2

失败数

0

创建时间

2026-10-16 15:52:55

开始时间

2026-10-16 15:52:55

结束时间

2026-10-16 15:52:55

最近错误

无



未覆盖项

- 前端没有测试脚本。`npm test` 不存在。页面验收靠 `npm run lint`、`npm run build` 和创建、详情、错误明细的手工走查，不在这次 `pytest` 里。
- 10MB 用例检查的是创建接口的字节上限，不把这 10MB 内容当合法 CSV 去解析。行数上限另有 10000 / 10001 用例。
- 幂等键、自动重试、取消和导出没有实现，也没有用例。
- 批量写入失败断言的是 `INTERNAL_ERROR` 和批号文案，没有单独的 `DB_WRITE_FAILED`。
- 超时回收只测了 Worker 启动时的一次。没有周期扫描，也没有断言回收时删除已提交的 `sync_records`。
- 宿主机 Worker 和 Compose Worker 同时消费同一队列的路径不一致，没有自动化用例。运行时不要同时开两个消费者。

来源文件：README.md（本地部署说明）

SyncFlow

SyncFlow 提供一个可追踪的数据导入闭环：用户通过 React 页面提交 CSV，接口立刻创建任务并返回，Worker 从 Redis 取出任务，校验每一行，把合法数据写入 MySQL，并保存状态、统计和错误明细。

1. 技术栈

层级	选型
后端 / Worker	Python 3.13 + FastAPI + Uvicorn
Web	React 19 + TypeScript + Vite + React Router
数据库	MySQL 8.0（镜像 `mysql:8.0`）
队列	Redis 7（镜像 `redis:7-alpine`）
编排	Docker Compose
配置	`.env` / `.env.example`

2. 环境要求

- Python 3.13。创建虚拟环境用 `py -3.13`，不要用其它版本。
- Node.js 20+
- Git
- Docker Desktop。Windows 上先启用 WSL2，并确认 Docker 状态是 Running。

3. 仓库结构

```
syncflow/
├── backend/                  # FastAPI 与 Worker（python -m app.worker）
├── frontend/                 # React + Vite
├── docs/                     # API 说明、周任务、发布记录
├── samples/                  # 可复现的 CSV 样例和期望统计
├── scripts/                  # 迁移脚本
├── uploads/                  # 上传目录，文件不提交
├── docker-compose.yml
├── .env.example
└── README.md
```

字段规则和 curl 示例在 docs/api.md。样例期望统计在 samples/README.md。

4. CSV 契约

文件必须是 `.csv`，UTF-8（允许 BOM），逗号分隔，第一行是表头。空行忽略，不计入总数。默认最大 10MB，对应 `MAX_UPLOAD_FILE_SIZE_MB`。单个任务默认最多 10000 条有效数据行，对应 `MAX_RECORDS_PER_JOB`。正好等于上限可以通过，多 1 字节或多 1 行会被拒绝。

表头必须且只能是下面四列，顺序固定：

```
external_id,name,amount,record_date
```

字段	类型	必填	校验	存储
external_id	string	是	去掉首尾空格后长度 1–64，只允许字母、数字、下划线和短横线；同一任务内大小写不敏感地唯一	大写
name	string	是	去掉首尾空格后长度 1–128	去掉首尾空格
amount	decimal(12,2)	是	非负，最多两位小数，不用科学计数法。`19`、`19.9`、`19.90` 都合法	`DECIMAL(12,2)`
record_date	date	是	严格 `YYYY-MM-DD`，且是真实日期	`DATE`

终态：全部成功是 `SUCCESS`；有成功也有行级失败是 `PARTIAL_SUCCESS`；文件级失败或全部行失败是 `FAILED`。`total_records = success_records + failed_records`。文件级失败时三个数都是 0。12 个 CSV 错误码见 docs/api.md。

5. 配置

```
Copy-Item .env.example .env
```

Linux 或 macOS：

```
cp .env.example .env
```

在宿主机跑 API 和 Worker 时，把 `.env` 改成：

```
MYSQL_HOST=127.0.0.1
REDIS_ADDR=127.0.0.1:6379
```

Compose 里的 API 和 Worker 会自己覆盖这两项，`.env` 保持 `mysql` 和 `redis:6379` 即可。

`MAX_RECORDS_PER_JOB` 默认 `10000`，Worker 进程启动时读取。`MAX_UPLOAD_FILE_SIZE_MB` 默认 `10`。`JOB_TIMEOUT_SECONDS` 默认 `300`，Worker 只在启动时回收超时的 `RUNNING` 任务。

`MAX_JOB_ATTEMPTS`、`RETRY_BACKOFF_SECONDS`、`IDEMPOTENCY_KEY_TTL_HOURS` 先留在 `.env` 里，V1.0 不使用。

上传目录不要写进 `.env`。Compose 把 `./uploads` 挂到容器内的 `/data/uploads`。宿主机进程必须在自己的终端里设置 `UPLOAD_DIR`，见下一节。不要同时跑宿主机 Worker 和 Compose Worker：两边看到的文件路径不一样，后启动的那个会把任务标成 `FILE_UNREADABLE`。

6. 本地启动

在仓库根目录执行。

6.1 后端虚拟环境

```
py -3.13 -m venv backend\.venv
backend\.venv\Scripts\Activate.ps1
pip install -r backend\requirements.txt
```

6.2 MySQL 和 Redis

```
docker compose up -d mysql redis
```

等 `docker compose ps` 里 mysql、redis 都是 healthy，再迁移：

```
powershell -ExecutionPolicy Bypass -File .\scripts\migrate.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\migrate.ps1 -Test
```

两条命令应分别打印 `migrated: syncflow` 和 `migrated: syncflow_test`。

6.3 API

新开一个终端，位于仓库根目录：

```
$env:UPLOAD_DIR = (Resolve-Path .\uploads).Path
backend\.venv\Scripts\Activate.ps1
Set-Location backend
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

如果 Windows 把 8000 划进了排除端口，API 会绑不上。换一个端口，例如 18000，并在前端终端设置 `$env:API_PROXY_TARGET = "http://127.0.0.1:18000"`。不要为了绕过它去改已经提交的 `docker-compose.yml` 端口。

6.4 Worker

再开一个终端，同样在仓库根目录。必须和 API 使用同一个 `UPLOAD_DIR`：

```
$env:UPLOAD_DIR = (Resolve-Path .\uploads).Path
backend\.venv\Scripts\Activate.ps1
Set-Location backend
python -m app.worker
```

没有这个进程时，任务会停在 `PENDING`。

6.5 前端

```
Set-Location frontend
npm install
npm run dev
```

浏览器打开 http://127.0.0.1:5173 。创建任务的页面是 http://127.0.0.1:5173/jobs/new 。

6.6 只用 Compose

镜像可用、并且宿主机能绑定 8000 时，可以在根目录执行：

```
docker compose up -d --build
```

这种方式不要再另开宿主机 Worker。Web 容器通过 `API_PROXY_TARGET=http://api:8000` 访问 API。

7. 用样例做一次导入

先确认第 6 节的 API 和 Worker 都在运行，然后回到仓库根目录：

```
$created = curl.exe -s -X POST "http://127.0.0.1:8000/api/v1/jobs" -F "name=样例-正常" -F
"file=@samples/ok.csv;type=text/csv" | ConvertFrom-Json
$id = $created.data.id
Start-Sleep -Seconds 2
curl.exe -s "http://127.0.0.1:8000/api/v1/jobs/$id"
```

创建请求应马上返回，`data.status` 为 `PENDING`。Worker 跑完后详情里是 `SUCCESS`，`total_records`、`success_records`、`failed_records` 分别是 2、2、0。页面上也可以看到同样的数字。

其余样例的期望统计在 samples/README.md。把上面的文件名换成 `samples/row-errors.csv`、`samples/bad-header.csv`、`samples/bom.csv` 或 `samples/empty.csv` 即可。

8. 停止

在 API、Worker 和前端所在的终端按 Ctrl+C。数据库和队列：

```
docker compose down
```

9. 数据库迁移

业务表不由 MySQL 镜像自动创建。`scripts/init_db.sql` 通过迁移脚本应用到已经启动的库。脚本可以重复执行。

Linux：

```
docker compose up -d mysql
chmod +x scripts/migrate.sh
./scripts/migrate.sh
./scripts/migrate.sh --test
```

不传参数时使用 `syncflow`。`--test` 使用 `syncflow_test`，避免测试数据进开发库。

Windows 命令见 6.2。查看库和表之前，先把 `.env` 读进当前终端：

```
Get-Content .env | ForEach-Object {
  if ($_ -match '^\s*#' -or $_ -notmatch '=') { return }
  $name, $value = $_.Split('=', 2)
  Set-Item -Path "Env:$name" -Value $value
}
docker compose exec -T mysql mysql --user=$env:MYSQL_USER --password=$env:MYSQL_PASSWORD --host=127.0.0.1 -e "SHOW
DATABASES;"
docker compose exec -T mysql mysql --user=$env:MYSQL_USER --password=$env:MYSQL_PASSWORD --host=127.0.0.1 -e "SHOW
TABLES FROM $env:MYSQL_DATABASE; SHOW TABLES FROM $env:MYSQL_TEST_DATABASE;"
```

每个库都应有且只有 `sync_jobs`、`sync_records`、`sync_errors`。

```
docker compose exec -T mysql mysql --user=$env:MYSQL_USER --password=$env:MYSQL_PASSWORD --host=127.0.0.1
$env:MYSQL_DATABASE -e "SHOW CREATE TABLE sync_jobs\G; SHOW CREATE TABLE sync_records\G; SHOW CREATE TABLE
sync_errors\G;"
```

对象	应该看到
`sync_jobs`	主键 `id`；索引 `(status, created_at)`、`(created_at)`
`sync_records`	`amount` 为 `decimal(12,2)`；`UNIQUE (job_id, external_id)`
`sync_errors`	索引 `(job_id, row_number)`；`raw_row` 为 `json`
字符集	`DEFAULT CHARSET=utf8mb4`

账号权限使用 `.env` 里的 root 密码：

```
docker compose exec -T mysql mysql --user=root --password=$env:MYSQL_ROOT_PASSWORD --host=127.0.0.1 -e "SHOW GRANTS
FOR '$($env:MYSQL_USER)'@'%';"
```

里面应有对 `syncflow` 和 `syncflow_test` 的授权。再执行一次 6.2 的两条迁移命令，仍应打印 `migrated:`，表还是这三张。

10. 前端构建

```
Set-Location frontend
npm install
npm run build
```

构建产物在 `frontend/dist`。本地开发用 `npm run dev`，不用先构建。

11. 测试

后端测试写入 `syncflow_test`，不写开发库 `syncflow`。在 `backend` 目录执行：

```
Set-Location backend
.\.venv\Scripts\python.exe -m pytest
```

前端没有 `npm test`。检查用：

```
Set-Location frontend
npm run lint
npm run build
```

探活：

```
curl.exe -s http://127.0.0.1:8000/healthz
curl.exe -s http://127.0.0.1:8000/readyz
```

12. 查看日志

Compose 服务：

```
docker compose logs -f api
docker compose logs -f worker
docker compose logs -f mysql
docker compose logs -f redis
docker compose logs -f web
```

按第 6 节在宿主机启动时，日志就在对应终端里。导入日志的 logger 名是 `syncflow.import`。

13. 清理数据

下面的命令会删掉 MySQL 数据卷和 Redis 容器数据：

```
docker compose down -v
```

清完后需要重新执行 6.2 的迁移。上传文件在 `uploads/`，按文件名手工删除即可，不要删 `uploads/.gitkeep`。

14. 已知限制

- 幂等键、自动重试和导出接口不属于 V1.0。
- 批量写入失败使用 `INTERNAL_ERROR`，不另建错误码。
- 错误明细里空的 `row_number` 会排在有行号的记录前面。
- 超时回收只在 Worker 启动时做一次。
- 本机若不能绑定 8000，用 6.3 的备用端口，不要改 Compose 文件。

来源文件：docs/week-03/Release-Notes-v1.0.md（Release Notes）

SyncFlow V1.0

发布日期：2026-10-16。Git 标签：`v1.0.0`（V1.0 发布提交）。

版本概述

用户在 React 页面选择 CSV 并创建任务。创建接口把任务 ID 放进 Redis 后立即返回，不等待整份文件处理完。Worker 读取 UTF-8 CSV（兼容 BOM），校验表头和每一行，把合法记录写入 MySQL `sync_records`，把失败写入 `sync_errors`，并在同一个事务里更新总数、成功数、失败数和终态。

终态有三种：`SUCCESS`、`PARTIAL_SUCCESS`、`FAILED`。详情页在任务还没结束时每 3 秒刷新，结束后停止。失败数大于 0，或者状态已经是 `FAILED`，都可以打开错误明细。文件级错误没有行号，页面显示 `-`。

安装说明

按仓库根目录 `README.md` 第 6 节，从 `.env.example` 开始安装依赖、启动 MySQL 和 Redis、迁移两套数据库、再启动 API、Worker 和前端。然后按第 7 节上传 `samples/ok.csv`。创建响应里的状态是 `PENDING`；Worker 完成后详情是 `SUCCESS`，统计为 2 / 2 / 0。

其它样例的期望统计在 `samples/README.md`。测试命令和 65 个用例的结果在 `docs/week-03/测试报告.md`。接口示例在 `docs/api.md`。

规范

- 表头只有 `external_id,name,amount,record_date`。金额用 `Decimal`，不用 float。
- 文件级失败的 `total_records`、`success_records`、`failed_records` 都是 0。
- 批量写入失败复用 `INTERNAL_ERROR`，消息里带批号。原始数据库报错只写日志。
- 错误明细排序是 `row_number ASC, id ASC`。空行号排在前面。错误码表见api.md.
- Worker 只在启动时回收超时的 `RUNNING` 任务，消息是「任务处理超时」。意外失败的消息是「Worker 处理失败」。已经结束的任务不会被再次标成失败。

不包含

幂等键、失败自动重试、取消任务、导出错误文件。`.env` 里的 `MAX_JOB_ATTEMPTS`、`RETRY_BACKOFF_SECONDS`、`IDEMPOTENCY_KEY_TTL_HOURS` 保留但未接入。这些留在 V1.1 的议题里，不在这个标签上改主线行为。

使用注意

不要同时运行宿主机 Worker 和 Compose Worker。两边的上传目录路径不同。改 `MAX_RECORDS_PER_JOB` 后要重启 Worker，演示结束后改回 `10000`。Windows 若不能绑定 8000，按 README 换端口，不要改 Compose 的端口映射。

测试环境

项目只在一下环境测试过：

系统环境	版本
Python	3.13.2
MySQL	8.0.46
Redis	7.4.11
操作系统	Windows11Pro 26H2
pytest	9.1.1
Docker	29.8.1
Docker Compose	v5.5.1

相关文档索引

下面按用途分组。路径都以仓库根目录为基准，链接相对本文件。

发布与验收

文档	仓库路径	内容
V1.0 测试报告	`docs/week-03/测试报告.md`	测试环境、执行命令、`65 passed` 结果和逐条用例清单
AI Code Review 记录	`docs/week-03/AI-Code-Review.md`	对照任务书与任务计划，检查已合并主线和发布提交
三分钟演示脚本	`docs/week-03/演示脚本.md`	演示前的启动步骤与逐段时间线
V1.0 复盘初稿	`docs/week-03/复盘初稿.md`	本期完成范围、遗留议题与 V1.1 输入

接口与设计

文档	仓库路径	内容
API 文档	`docs/api.md`	已实现的 HTTP 接口、请求响应示例与错误码表
API 设计（第 1 周）	`docs/week-01/05-API设计.md`	接口清单与字段设计的源头规格
数据库设计	`docs/week-01/06-数据库设计.md`	`jobs` / `sync_records` / `sync_errors` 表结构与索引
任务状态机	`docs/week-01/07-任务状态机.md`	`PENDING` / `RUNNING` 与三种终态的迁移规则
Web 页面设计	`docs/week-01/08-Web页面设计.md`	创建、详情刷新、错误明细三个页面
业务名词表	`docs/week-01/02-业务名词表.md`	术语口径，避免统计与错误命名歧义
需求疑问清单	`docs/week-01/01-需求疑问清单.md`	第 1 周澄清记录
测试计划（第 1 周）	`docs/week-01/09-测试计划.md`	异常场景、边界值与期望结果
系统边界图	`docs/week-01/03-系统边界图.png`	系统边界示意
核心流程图	`docs/week-01/04-核心流程图.png`	CSV 同步主链路

需求来源

文档	仓库路径	内容
产品需求文档	`docs/PRD/product-requirements.pdf`	PRD 原文
第 1 周任务书	`docs/tasks/week-01.pdf`	需求与设计周任务
第 2 周任务书	`docs/tasks/week-02.pdf`	骨架搭建周任务
第 3 周任务书	`docs/tasks/week-03.pdf`	CSV 同步闭环与 V1.0 发布周任务

在线文档：服务启动后 `http://127.0.0.1:8000/docs` 是交互式 API 文档，`http://127.0.0.1:8000/openapi.json` 是机器可读规格。

来源文件：docs/week-03/演示脚本.md（演示脚本）

项目演示脚本

基本信息

- 项目名称：SyncFlow —— CSV 批量导入与任务管理服务
- 演示版本 / Git commit：V1.0，标签 `v1.0.0`
- 预计时长：5 分钟
- 演示范围：CSV 导入闭环 —— 创建任务立即返回 → Worker 异步处理 → 成功统计 → 行级错误与文件级错误明细 → 现场改一处业务规则 → 测试证据

录制前准备

- [ ] 准备样例文件：`samples/ok.csv`、`samples/row-errors.csv`、`samples/bad-header.csv`（可选 `samples/bom.csv`、`samples/empty.csv`）
- [ ] 确认依赖环境可用：MySQL、Redis、API、Worker、前端五个服务都在跑，`docker compose -f docker-compose.yml -f docker-compose.e2e.yml ps` 里 mysql / redis 为 healthy，api / worker / web 为 Up
- [ ] 确认端口：**本机 8000 落在 Windows 保留区间（`netsh interface ipv4 show excludedportrange protocol=tcp` 显示 7998–8097），所以统一用 18000。**Compose 走覆盖文件：

```
docker compose -f docker-compose.yml -f docker-compose.e2e.yml up -d --build
```

只有宿主机能绑定 8000 时，才按 `README.md` 6.3 用 8000 起宿主机 API 与 Worker

- [ ] 确认只留**一个** Worker：宿主机 Worker 与 Compose Worker 同时消费 `syncflow:jobs` 会把任务标成文件不可读，二者只能留一个
- [ ] 确认 `UPLOAD_DIR` 一致：宿主机起 API/Worker 时都要设 `$env:UPLOAD_DIR = (Resolve-Path .\uploads).Path`；Compose 走容器内的 `/data/uploads`，两边路径不同
- [ ] 确认 `.env` 的 `MAX_RECORDS_PER_JOB` 为 `10000`、`MAX_UPLOAD_FILE_SIZE_MB` 为 `10`
- [ ] 清理上一次演示结果：列表页不留旧任务，`uploads/` 里的历史文件按需删除（保留 `.gitkeep`）
- [ ] 隐藏敏感信息并关闭通知：`.env` 含数据库口令，投屏前不要停留在该文件；关闭 IM / 邮件弹窗
- [ ] 准备对应版本的测试报告：`docs/week-03/测试报告.md`（65 passed）与 `docs/week-03/screenshots/` 截图备用
- [ ] 确认浏览器停在 `http://127.0.0.1:5173/jobs/new`，缩放 100%，书签栏收起
- [ ] 按脚本完整预演一次；预演结束后把 `MAX_RECORDS_PER_JOB` 改回 `10000` 并重启 Worker

2. 分镜总览

编号	时间	环节	主要画面与操作	展示目的
S01	00:00–00:20	项目介绍	展示项目名称、版本标签与任务详情页	说明项目目标与本次范围
S02	00:20–00:45	环境与数据	展示 `docker compose ps`、探活接口、`samples/` 目录	明确演示条件与样例口径
S03	00:45–01:35	创建并立即返回	选择 `ok.csv` 提交，展示 `PENDING` 与接口耗时	证明创建不等待整份 CSV
S04	01:35–02:15	成功统计	详情页终态 2 / 2 / 0，列表页筛选，库内 `sync_records` 两行	证明数据真正落地且统计准确
S05	02:15–03:10	行级与文件级错误	上传 `row-errors.csv` 与 `bad-header.csv`，查看错误明细	证明非法数据可定位到行与原因
S06	03:10–03:55	现场改一处规则	改 `MAX_RECORDS_PER_JOB=1` 重启 Worker，再传 `ok.csv`	证明规则集中在一处、可现场改动
S07	03:55–04:25	测试与验证	展示 `pytest` 结果与端到端截图	说明验证方式与覆盖范围
S08	04:25–05:00	总结与限制	展示已完成项、已知限制与 V1.1 计划	明确成果边界与后续方向

3. 详细分镜脚本

S01：项目介绍

- 时间：00:00–00:20
- 画面：任务列表页 `http://127.0.0.1:5173`，标题与「创建任务」按钮可见。

操作步骤

1. 展示项目名称 SyncFlow 与页面顶部导航。
2. 打开 `README.md` 首行的版本标签说明，指出本次是 `v1.0.0`。
3. 切换到任务详情页，让观众先看到统计与「查看错误明细」入口的最终形态。

预期画面 / 结果

- 项目名称、版本 `v1.0.0` 清晰可见。
- 本次演示范围明确：只讲导入闭环，不讲登录、重试、导出。

S02：环境与数据

- 时间：00:20–00:45
- 画面：终端 A（Compose 状态）、终端 B（探活）、资源管理器或编辑器里的 `samples/` 目录。

操作步骤

1. 在仓库根目录执行 `docker compose -f docker-compose.yml -f docker-compose.e2e.yml ps`，指出 mysql、redis 为 `healthy`，api、worker、web 为 `Up`，api 的端口是 `0.0.0.0:18000->8000/tcp`。
2. 执行探活：

```
curl.exe -s http://127.0.0.1:18000/healthz
curl.exe -s http://127.0.0.1:18000/readyz
```

指出 `readyz` 的 `data.checks` 里 mysql 与 redis 都是 `ok`（实测返回 `{"status":"ok","checks":{"mysql":"ok","redis":"ok"}}`）。

1. 展示 `samples/` 下五份样例与 `samples/README.md` 的期望统计表。

预期画面 / 结果

- 五服务状态正常，探活返回 ok。
- 观众知道“期望统计”不是临场报数，而是仓库里已有的口径。

S03：创建并立即返回

- 时间：00:45–01:35
- 画面：创建任务页 `/jobs/new` 与提交后的详情页。

操作步骤

1. 打开 `http://127.0.0.1:5173/jobs/new`，选择 `samples/ok.csv`，指出页面出现「已选择 ok.csv（100 B）」，文件名与大小都对。
2. 任务名称填 `演示-正常`，点击提交。提交期间指出按钮文案变为「正在创建…」且不可点。
3. 请求返回后进入详情页，指出状态徽标是「等待中」或很快变成「处理中」。
4. 在终端补一条计时命令，展示创建接口的响应耗时：

```
$t0 = Get-Date
$ok = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
  -F "name=演示-正常" -F "file=@samples/ok.csv;type=text/csv" | ConvertFrom-Json
"POST 耗时(ms) = " + [int]((Get-Date) - $t0).TotalMilliseconds
"创建时状态 = " + $ok.data.status
$okId = $ok.data.id
```

指出状态是 `PENDING`，耗时在百毫秒量级。这条命令会另建一个任务，说明它就是刚才页面那次调用（端到端实测 196 ms，最近复测 108 ms，数量级一致即可，不必念具体数字）。

预期画面 / 结果

- 创建响应立即返回，`data.status` 为 `PENDING`。
- 详情页显示「正在处理，每 3 秒自动刷新。」
- 页面没有卡在上传等待上。

S04：成功统计

- 时间：01:35–02:15
- 画面：详情页终态、任务列表页筛选、MySQL 查询结果。

操作步骤

1. 等详情页变成「已完成」，指出总数 2、成功 2、失败 0，结束时间已有，且**没有**「查看错误明细」入口。
2. 回到列表页 `http://127.0.0.1:5173/`，把状态筛选切到「已完成」，指出新任务出现在列表里。
3. 在终端查库，核对写入的两行（`$okId` 就是第 4 步拿到的任务 id；若只走了页面没用终端，把 `$okId` 换成页面上那条任务的 id）：

```
  docker compose exec -T mysql mysql --user=syncflow --password=syncflow123 --host=127.0.0.1 syncflow -e "SELECT job_id,external_id,name,amount,record_date FROM sync_records WHERE job_id='$okId';"
```

指出 `external_id` 为大写、金额是两位小数、日期是 `2026-01-01`。

预期画面 / 结果

- 详情页与列表页都显示 SUCCESS、2 / 2 / 0。
- `sync_records` 恰好两行，字段值与样例期望一致。
- 成功任务没有「查看错误明细」入口（反向断言）。

S05：行级与文件级错误

- 时间：02:15–03:10

- 画面：错误明细页 `/jobs/:jobId/errors`，两张不同的错误表格。

操作步骤

1. 回到创建页上传 `samples/row-errors.csv`，等终态为「部分成功」，指出总数 8、成功 1、失败 7。
2. 点「查看错误明细」，指出行号 3 到 9 与逐行错误码：`FIELD_REQUIRED`、`EXTERNAL_ID_INVALID`、`EXTERNAL_ID_DUPLICATE`、`NAME_TOO_LONG`、`AMOUNT_INVALID`、`DATE_INVALID`、`ROW_COLUMN_COUNT_MISMATCH`。
3. 再上传 `samples/bad-header.csv`，指出终态是「失败」，三个计数都是 0，但**仍然**有「查看错误明细」入口。
4. 打开明细，指出行号显示 `-`、错误码是 `CSV_HEADER_INVALID`；顺便说明 `samples/empty.csv` 同理，码是 `FILE_EMPTY`。
5. （可选）切换到 `bom.csv`，说明带 UTF-8 BOM 的文件同样能导入成功。

预期画面 / 结果

- `row-errors.csv`：`PARTIAL_SUCCESS`、8 / 1 / 7、7 条明细、错误码逐个对上。
- `bad-header.csv`：`FAILED`、0 / 0 / 0、1 条明细、行号为 `-`。
- 观众能看出“文件级失败”和“行级失败”的区别。

S06：现场改一处规则

- 时间：03:10–03:55
- 画面：`.env` 编辑器、Worker 终端、创建页与失败详情页。

操作步骤

1. 打开 `.env`，把 `MAX_RECORDS_PER_JOB=10000` 改成 `1`，保存（投屏时不要展示同文件里的口令行）。
2. 重启 Worker，让它重新读配置。**Compose 路径**：

```
docker compose -f docker-compose.yml -f docker-compose.e2e.yml up -d worker
```

用 `up -d` 而不是 `restart`：`env_file` 变了要重建容器，`restart` 沿用旧环境变量。宿主机路径则在 Worker 终端 Ctrl+C 后重新 `python -m app.worker`。

1. 再上传 `samples/ok.csv`：

```
$big = curl.exe -s -X POST "http://127.0.0.1:18000/api/v1/jobs" `
  -F "name=演示-超上限" -F "file=@samples/ok.csv;type=text/csv" | ConvertFrom-Json
Start-Sleep -Seconds 3
(curl.exe -s "http://127.0.0.1:18000/api/v1/jobs/$($big.data.id)" | ConvertFrom-Json).data
```

指出这次终态变成「失败」，错误码 `FILE_TOO_MANY_ROWS`，计数仍是 0 / 0 / 0，且没有新增 `sync_records`。

1. 把 `MAX_RECORDS_PER_JOB` 改回 `10000`，用同一条 `up -d worker` 重建 Worker。**这一步必须当场做完**，否则后续后端测试会失败。

预期画面 / 结果

- 改动一行配置后，`ok.csv` 从 `SUCCESS` 变为 `FAILED` / `FILE_TOO_MANY_ROWS`。
- `sync_records` 没有写入新行。
- 演示结束时配置已恢复 `10000`，Worker 已重启。

S07：测试与验证

- 时间：03:55–04:25
- 画面：`pytest` 输出、`docs/week-03/测试报告.md`、`docs/week-03/screenshots/` 文件夹。

操作步骤

1. 在 `backend` 目录执行：

```
.\.venv\Scripts\python.exe -m pytest -q --tb=line
```

指出输出 `65 passed`。

1. 打开 `docs/week-03/测试报告.md`，翻到「用例清单」与「端到端验证」两节，指出 65 条用例逐条对应到 `文件::函数`。
2. 打开截图目录，快速划过 `链路搭建`、`探活`、`成功任务终态`、`失败任务`、`Worker日志`、`页面检查` 六组共 10 张，说明页面与链路都有留档。
3. 指出未覆盖项那一节：前端没有 `npm test`，页面验收靠 `npm run lint`、`npm run build` 加手工走查。

预期画面 / 结果

- `65 passed`，无失败无跳过。
- 报告与截图能对应上刚才演示的三个场景。
- 未覆盖项被主动说明，而不是被问到。

S08：总结与限制

- 时间：04:25–05:00
- 画面：任务列表页（展示刚才三种终态的任务）+ `docs/week-03/Release-Notes-v1.0.md` 的「不包含」一节。

操作步骤

1. 回到列表页，展示 `SUCCESS`、`PARTIAL_SUCCESS`、`FAILED` 三种终态同时存在。
2. 打开 Release Notes 的「不包含」与「使用注意」两节，指出幂等键、重试、取消、导出都不在 V1.0。
3. 打开 `docs/week-03/复盘初稿.md` 的「后续行动」表，指出 V1.1 的议题。

预期画面 / 结果

- 三种终态在列表页同屏可见，呼应前面每个分镜。
- 已完成项与未完成项分开陈述，边界清晰。
- 观众知道 V1.1 的入口在复盘文档里，不在本次演示的承诺里。

结尾

- 本次已验证：
  - 创建任务接口立即返回 `PENDING`，端到端实测 196 ms，不等待整份 CSV。
  - 合法数据写入 MySQL：`ok.csv` 为 `SUCCESS` 2 / 2 / 0，`sync_records` 两行与 `samples/README.md` 一致。
  - 非法数据可定位：`row-errors.csv` 为 `PARTIAL_SUCCESS` 8 / 1 / 7 并逐行给出错误码；`bad-header.csv` 与 `empty.csv` 为文件级失败，行号显示 `-`。
  - 业务规则集中在 `MAX_RECORDS_PER_JOB`，改一处配置即可把同一份文件从成功变为 `FILE_TOO_MANY_ROWS`。
  - `pytest` 65 passed，端到端链路与页面检查均有截图留档。
- 当前限制：
  - 不实现登录与权限；不实现失败重试、幂等键、任务取消和错误导出。
  - `WORKER_CONCURRENCY` 未接入，Worker 单进程串行消费，1,000 行并发基线未验证。
  - 超时回收只在 Worker 启动时执行一次，且会误伤 `started_at` 为空的 `RUNNING`。
  - 错误明细排序为 `row_number ASC, id ASC`，空行号（文件级错误）排在最前。
  - 前端没有自动化测试，页面验收靠 `npm run lint`、`npm run build` 与手工走查。
  - 本机宿主端口 8000 落在 Windows 保留区间时，Compose 需用 `docker-compose.e2e.yml` 覆盖为 18000。
- 下一阶段计划：
  - 为前端补关键交互测试（创建、列表、详情三态与轮询停止），见 `docs/week-04/前端测试计划.md`。
  - 接入 `WORKER_CONCURRENCY` 并补 1,000 行并发基线用例。
  - 评估超时回收改为周期执行，并明确回收后已写入记录与计数的处置。
  - 按复盘行动项开启 V1.1 议题：重试策略与执行记录、幂等键与文件摘要、任务取消、CI 自动检查、结果导出。

相关文档

文档	路径	用途
测试报告	`docs/week-03/测试报告.md`	S07 展示的 65 个用例与端到端验证记录
Release Notes	`docs/week-03/Release-Notes-v1.0.md`	S08 展示的范围、不包含项与文档索引
复盘初稿	`docs/week-03/复盘初稿.md`	S08 展示的后续行动表
样例与期望统计	`samples/README.md`	S02、S04、S05 的数字来源
部署与启动说明	`README.md`	录制前准备第 6 节、测试第 11 节、已知限制第 14 节
API 文档	`docs/api.md`	被追问接口细节时的应答依据
端到端截图	`docs/week-03/screenshots/`	S07 展示的 10 张留档

来源文件：docs/week-03/复盘初稿.md（复盘初稿）

项目阶段复盘

1. 基本信息

- 项目：SyncFlow 数据同步与任务管理服务
- 复盘范围 / 版本：V1.0（第 1–3 周）。CSV 同步闭环与 V1.0 发布包
- 开发周期：2026-09-21 ~ 2026-10-16（3 周，26 个提交）
- 复盘日期：2026-10-16
- 参与人员：1 人

2. 背景与目标

背景

业务人员需要把 CSV 里的商品、订单或运营数据批量导入业务系统。人工录入效率低、容易遗漏；一次性脚本导入虽然快，但通常缺少任务状态、错误明细和重复导入保护。导入失败后，使用者很难判断是文件格式问题、某一行数据不合法，还是系统暂时不可用。

SyncFlow 要提供一条可追踪的导入闭环：用户在 React 页面提交 CSV，系统创建任务并异步处理，逐行校验，把合法数据写入 MySQL，同时保存状态、统计和错误明细。

主要约束：

- 本地可运行：`docker compose up` 拉起 Web / API / Worker / MySQL / Redis 五个服务，核心功能不建在云服务、外部数据库或个人账号上。
- 技术栈统一：后端 Python + FastAPI 或 Go + Gin 二选一（本项目选 Python + FastAPI）；前端 React 18+ / TypeScript / Vite / React Router；MySQL 8.x；Redis 7.x。
- 数据原则：金额用 DECIMAL 不用浮点；时间用 `datetime(3)` 存 UTC；表间不建外键、关联在应用层维护；同一任务内业务标识唯一。
- 接口原则：响应统一信封；不返回内部路径、SQL、堆栈和连接配置；列表接口必须分页。
- 范围：不实现登录与权限，所有功能对所有访问者开放。
- 协作：所有变更走分支 → PR → AI Code Review，不直接推主分支。

原定目标

出自第 3 周「V1.0 发布标准」，共 7 条：

1. 用户可以通过 React Web 页面创建任务和查看结果。验收：浏览器可完成创建→详情→错误明细全流程。
2. 创建任务接口不会等待整个 CSV 处理完成。验收：创建响应立即返回，状态为 `PENDING`，处理随后由 Worker 完成。
3. 合法 CSV 数据能够写入 MySQL。验收：`sync_records` 行数与样例期望一致。
4. 非法数据能够查询到行号和错误原因。验收：错误明细接口与页面能看到 `row_number` 和 `error_code`。
5. 任务统计数据准确。验收：样例期望统计 = 页面 / 接口 / 表内统计。
6. 代码经过至少一次 PR 和 AI Code Review。验收：有可独立审阅的 PR 与 Review 记录。
7. 能现场解释并修改一处业务规则。验收：规则集中在一处，演示时改一个配置就能改变行为。

三项产出物（第 3 周「产出物细则」）：

- 异步导入闭环：UTF-8/BOM、表头与字段校验、去重、Redis 投递、Worker 消费、批量事务、统计与错误记录。判定：创建接口快速返回；Worker 可将任务处理至终态；成功和失败数据分别在 MySQL 可查。
- 完整 Web 操作页面：CSV 选择与前端校验、上传状态、任务详情刷新、错误表格和分页。判定：合法文件可创建；非法文件无法提交；运行中状态和错误明细均可在页面查看。
- V1.0 发布包：正常/异常样例、测试报告、部署说明、Release Notes、演示和复盘。判定：空环境可复现版本；样例预期与实际统计一致。

3. 实际交付结果

目标	实际结果	状态	证据 / 链接
React 页面创建任务与查看结果	创建页 `/jobs/new`、列表页 `/`、详情页 `/jobs/:jobId`、错误明细页 `/jobs/:jobId/errors` 全部可用；详情页非终态每 3 秒轮询、终态自动停止	已完成	测试报告·页面检查、页面检查-成功、页面检查-错误
创建接口不等待整份 CSV	端到端实测 POST 耗时 196 ms，返回 `status=PENDING`，Worker 随后处理至终态	已完成	测试报告·端到端验证、成功任务
合法 CSV 写入 MySQL	65 个用例全通过；端到端 `ok.csv` 写入 `sync_records` 2 行，统计 2 / 2 / 0	已完成	测试报告·结果、成功任务终态
非法数据可查行号与原因	行级 7 个错误码逐条可定位；文件级错误 `row_number` 为空、页面显示 -	已完成	测试报告·用例清单、失败任务

目标	实际结果	状态	证据 / 链接
任务统计准确	五份样例的 `total / success / failed` 与 `samples/README.md` 期望逐项一致，由测试锁住	已完成	samples/README.md、测试报告·样例用例
PR 与 AI Code Review	5 个 PR 已分模块合并到主线；本地 Review 记录已成文。远端 PR 上的 `/review` 尚未触发，见问题 2	未验证	AI-Code-Review.md、deepseek-review.yml
现场可改一处业务规则	规则集中在 `MAX_RECORDS_PER_JOB`：解析、导入、Worker 都读它；改值并重启 Worker 即可让 `ok.csv` 变为 `FILE_TOO_MANY_ROWS`	已完成	演示脚本、测试报告·用例清单
异步导入闭环	CSV 读取与校验、Redis 投递、Worker 消费、批量事务写入、统计与错误记录全部打通	已完成	测试报告·端到端验证、Worker日志
完整 Web 操作页面	四个页面齐备；加载中、空数据、请求失败三态有处理；未匹配路由显示 404	已完成	Web 页面设计、测试报告·页面检查
V1.0 发布包	5 份样例、测试报告、README、`.env.example`、API 文档、Release Notes、演示脚本、复盘	已完成	Release-Notes-v1.0.md、测试报告
本地五服务可启动	Compose 实测 `up 7/7`：mysql、redis Healthy，api、worker、web Started	已完成	链路搭建01、链路搭建02
前端关键交互自动化测试	只有 `npm run lint` 和 `npm run build`，无测试脚本；页面验收靠手工走查。差距见问题 3	延期	测试报告·未覆盖项
1,000 行并发基线	无对应用例，且 `WORKER_CONCURRENCY=2` 未接入，单进程 `BRPOP` 消费。见问题 6	延期	第 1 周测试计划

4. 做得好的地方

业务规则集中到单一配置与单一常量表

- 带来的收益：表头、字段规则、12 个错误码都定义在一处，页面、Worker 和测试不会各写一套；演示时改 `MAX_RECORDS_PER_JOB` 一个值就能改变终态，不用碰校验代码。
- 支持证据：`backend/app/core/error_codes.py` 集中定义文件级 5 码与行级 7 码，由 `test_sync_error_codes_match_week3_spec` 锁住集合与 `field_name`；`Settings.max_records_per_job` 被解析、导入和 Worker 共用，`test_max_records_per_job_reads_env` 验证默认为 10000 且读取环境变量。
- 后续如何复用：V1.1 的导出功能直接复用这套错误码，避免导出口径与页面不一致。

创建与导入分离，用队列解耦

- 带来的收益：创建接口只做校验、落盘、写 `PENDING` 和入队就返回，实测 196 ms；10,000 行的解析和写入在 Worker 里做，用户不必等整个文件算完。
- 支持证据：端到端 POST 返回 `PENDING`，随后同一任务在库中变为 `SUCCESS` 2 / 2 / 0；`test_create_job_success` 断言创建只返回 `id/name/status/created_at` 并已入测试队列。
- 后续如何复用：V1.1 加取消和重试时，仍从这条队列边界切入，不改创建接口契约。

样例数据 + 期望统计当作可执行的验收口径

- 带来的收益：`samples/` 五份文件各带期望的终态与计数，`test_samples.py` 直接对照断言，发布判定不用靠人肉核对页面数字。
- 支持证据：`ok.csv`、`bom.csv`、`row-errors.csv`、`bad-header.csv`、`empty.csv` 的终态与 `total/success/failed` 全部与 `samples/README.md` 一致；`row-errors.csv` 第 3–9 行的错误码逐行锁定。
- 后续如何复用：V1.1 的新功能沿用「加一份样例、写期望、进测试」的做法。

批量事务策略只回滚失败的那一批

- 带来的收益：一批唯一约束冲突不会让整个任务全废，其余批次的数据仍然落库，统计如实反映成功与失败，且消息里保留批号便于定位。
- 支持证据：`test_failed_batch_does_not_roll_back_other_batches` 断言 4 行中冲突 1 批时得到 `PARTIAL_SUCCESS` 与 2 / 2；`test_multiple_batch_failures_summarize_batch_numbers` 验证多批汇总文案。
- 后续如何复用：批大小 200 落在规格建议的 100–500 区间，后续调参只改 `RECORD_BATCH_SIZE`。

错误信息对外脱敏有明确边界

- 带来的收益：公开响应里不出现连接串、SQL、堆栈和内部路径，数据库原文只进日志，排障靠日志、对外靠稳定错误码。
- 支持证据：`test_database_error_is_hidden` 断言 500 响应不含 `secret-password`、`mysql+pymysql` 和 `Traceback` 且删除已写文件；`test_worker_unexpected_error_does_not_stay_running` 断言意外异常对外只有「Worker 处理失败」。
- 后续如何复用：V1.1 新增接口沿用同一信封与脱敏约定。

5. 问题与原因

问题 1：宿主端口 8000 落在 Windows 保留区间，Compose 的 API 无法发布

- 事实：`netsh interface ipv4 show excludedportrange protocol=tcp` 显示保留区间包含 7998–8097，8000 在其中。按 `docker-compose.yml` 原样 `8000:8000` 发布时，`syncflow-api-1` 停在 `Created` 从未启动，mysql、redis、worker 均已 Up。
- 影响：Compose 整条链路无法按默认端口起来，端到端验证一度无法进行；README 6.3 给的换端口方案只适用于宿主机进程，不覆盖 Compose。
- 原因与证据：Windows 的保留端口由系统与 Hyper-V 动态占用，与代码无关；`docker inspect syncflow-api-1` 显示 `created`，`TcpListener` 绑定 8000 返回拒绝访问，而 18000 可绑。
- 已采取的处理：新增 `docker-compose.e2e.yml`，用 `!override` 只把 API 的宿主端口改为 `18000:8000`，容器内端口与 web 的 `API_PROXY_TARGET=http://api:8000` 均不变；`docker-compose.yml` 未改动。端到端据此跑通并留档。
- 遗留问题：该文件已提交但 README 与 Release Notes 都未提及，读者不知道存在这条 Compose 路径；`docker-compose.yml` 的默认端口在同类机器上仍会失败。

问题 2：发布标签与完整发布包不一致

- 事实：`v1.0.0` 指向 `a256fbe`。标签里的 `docs/week-03/测试报告.md` 只有 61 行，没有「测试环境」和「端到端验证」两节，`docs/week-03/screenshots/` 在标签里为 0 个文件；工作区该报告已 173 行并含 10 张截图。`Release-Notes-v1.0.md` 在标签里也是更早的标题版本。
- 影响：Release Notes 让读者去测试报告看结果，但标签里那份没有端到端记录，第 3 周「V1.0 发布包」的「空环境可复现」判定缺少标签级证据。
- 原因与证据：端到端验证与报告扩充发生在打标签之后（截图时间 15:37–15:57，发布提交时间更早）；发布流程没有「标签前核对发布包完整性」这一步。
- 已采取的处理：把 Release Notes 首句从「指向包含本文件的发布提交」改为「（V1.0 发布提交）」，不再声称内容等同；端到端记录补进工作区报告。
- 遗留问题：标签是否重打或改发 `v1.0.1` 未决；截图目录目前仍是未跟踪状态。

问题 3：前端没有自动化测试

- 事实：`package.json` 只有 `dev`、`build`、`lint`、`preview` 四个脚本，没有 `test`；`npm test` 不存在。页面验收靠 `npm run lint`、`npm run build` 和手工走查。
- 影响：PRD 9.2 要求「前端至少覆盖创建任务、任务列表和任务详情的关键交互测试」，该条未达成；页面三态与轮询启停只能靠人看。
- 原因与证据：`frontend/` 下无 vitest/jest 配置，仅有 `eslint.config.js` 和 `vite.config.ts`；测试报告「未覆盖项」也记录了这点。
- 已采取的处理：暂时用手工走查加截图留档（创建、成功详情、文件级错误明细共 3 张页面截图）。
- 遗留问题：缺自动化回归手段，后续改页面容易打破三态或轮询停止逻辑而无人察觉。

问题 4：第 1 周设计文档与最终实现不一致，且未回填

- 事实：第 1 周设计文档用的是 `sku,name,price,quantity` 表头、`job_records / job_errors / job_attempts` 四张表和 `succeeded / partial_succeeded` 状态名；实现最终是 `external_id,name,amount,record_date`、`sync_jobs / sync_records / sync_errors` 三张表和 `SUCCESS / PARTIAL_SUCCESS`。API 设计里还列了 `/records`、`/cancel`、`/export` 和 `VALIDATION_ERROR`、`UNSUPPORTED_MEDIA_TYPE` 等本周未实现的路径与错误码。
- 影响：设计文档作为规格源头已经失真，新人按第 1 周文档读代码会对不上；评审时也无法用设计文档验证实现。
- 原因与证据：第 3 周任务书把表头和字段规则改成了 `external_id,name,amount,record_date`，实现跟的是新任务书，但第 1 周文档没有同步更新；对比 `docs/week-01/06-数据库设计.md` 与 `backend/app/models/`、`docs/week-01/05-API设计.md` 与实际路由即可看出。
- 已采取的处理：第 3 周的 `docs/api.md` 按实际实现重写了接口与错误码，作为当前权威口径。
- 遗留问题：第 1 周的 5 份设计文档仍在仓库中且未标注「已被第 3 周规格取代」。

问题 5：测试报告里的核对命令用了 MySQL 保留字

- 事实：测试报告「Worker」一节给的 `SELECT job_id,row_number,error_code FROM sync_errors ...` 会报 `ERROR 1064 (42000)`，因为 `ROW_NUMBER` 是 MySQL 8.0 保留字（`INFORMATION_SCHEMA.KEYWORDS` 中 `RESERVED=1`）。该报错已经出现在留档的 Worker 日志截图里。
- 影响：读者照抄该命令无法执行；那次端到端里查 `sync_errors` 的语句实际未成功，只有 `sync_jobs` 和 `sync_records` 两条查询返回了结果。
- 原因与证据：`ROW_NUMBER` 自 MySQL 8.0 起为保留字，未加反引号会被解析为窗口函数；PowerShell 双引号字符串中反引号又是转义符，直接写 ` row_number ` 会被吃掉。
- 已采取的处理：改用不加保留字的列集合（`job_id, field_name, error_code, error_message`），已实测通过。

问题 6：`.env` 中的并发与重试配置未接入

- 事实：`WORKER_CONCURRENCY=2`、`MAX_JOB_ATTEMPTS=3`、`RETRY_BACKOFF_SECONDS=5,30,120`、`IDEMPOTENCY_KEY_TTL_HOURS=24` 都在 `.env` 与 `.env.example` 里，但代码中没有任何一处读取。Worker 是单进程 `BRPOP` 串行消费。
- 影响：第 1 周测试计划的「1,000 行并发基线」无法验证；配置文件里存在「看起来生效其实没用」的项，容易误导运维。
- 原因与证据：全仓搜索这些变量只有 `.env` / `.env.example` / 文档命中，`backend/app/` 无引用；`worker.py` 主循环单个 `dequeue_job(timeout=5)`。
- 已采取的处理：在 Release Notes 的「不包含」和 README 第 14 节明确写出这些项保留但 V1.0 不接入。
- 遗留问题：配置项与实际能力的落差靠文档说明弥补，属于 V1.1 议题。

问题 7：超时回收只在 Worker 启动时执行一次，且会误伤 `started_at` 为空的 `RUNNING`

- 事实：reclaim_stale_running 在 Worker 启动时调用一次；筛选条件是 started_at IS NULL OR started_at < deadline，因此 started_at 为空的 RUNNING 会被立即收成 FAILED。回收只改 sync_jobs，不删已写入的 sync_records，也不重算计数。
- 影响：长时间运行中卡住的 RUNNING 不会被自动回收；被回收的任务统计可能与 sync_records 实际行数不一致。
- 原因与证据：worker.py 只在 main() 里调用一次 reclaim_stale_jobs；test_reclaim_stale_running_leaves_fresh_jobs 验证了超时与新鲜的区分，但没有覆盖 started_at 为空的边界。
- 已采取的处理：消息统一为「任务处理超时」，并把该限制写进 README 第 14 节与 Release Notes。
- 遗留问题：是否改为周期回收、以及回收后如何处置已写入记录，留给 V1.1。

问题 8：前端错误码常量文件无人引用

- 事实：frontend/src/api/errorCodes.ts 镜像了后端 12 个错误码与中文含义，但全 frontend/src 无任何 import，生产构建产物里也不含它。页面错误文案实际用的是接口返回的 error_message。
- 影响：同一套错误码存在两份定义，改后端容易漏改前端；这个文件让「前后端共用一套码」的说法看起来已落实，实际未生效。
- 原因与证据：grep errorCodes 只在该文件自身命中；npm run build 产出的 bundle 中搜不到其导出常量。
- 已采取的处理：明确以接口返回的 error_message 为准，并在测试报告与 AI Review 中记录。
- 遗留问题：要么接入页面做本地化兜底，要么删除该文件，二选一未定。

6. 关键决策与取舍

- **决策 1：文件级失败时 total / success / failed 三个计数都为 0。**
  - 当时的依据：第 3 周规格没有写死文件级失败时的计数口径，任务计划建议 total = success + failed，文件级错误只体现在 last_error_code 与 sync_errors。
  - 实际收益与代价：统计口径自洽，代价是「查看错误明细」入口不能只看 failed_records > 0，否则文件级失败点不进去。

- **决策 2：批量写入失败复用 INTERNAL_ERROR，不新增 DB_WRITE_FAILED。**
  - 当时的依据：规格给的 12 个错误码里没有「批量写入失败」，而要求记清哪批、什么原因；为不撑大错误码集合，复用已有接口错误码并在消息里带批号。
  - 实际收益与代价：错误码集合保持 12 个、与规格逐条对应；代价是 sync_errors 里出现了一个非 CSV 语义的码，语义上略有混杂。

- **决策 3：错误明细排序用 row_number ASC，id ASC，空行号排在最前。**
  - 当时的依据：规格只写了 row_number ASC，id ASC，未规定空行号位置；MySQL 默认把 NULL 排在前面，直接沿用即可。
  - 实际收益与代价：实现最简、与规格字面一致；代价是文件级错误出现在行级错误之前，顺序上略反直觉（通常想先看具体行）。

- **决策 4：不改已提交的 docker-compose.yml 端口，改用独立覆盖文件。**
  - 当时的依据：README 与复盘都明确「不要改已提交的 Compose 端口」；8000 被系统保留是环境问题，不该污染发布配置。
  - 实际收益与代价：主配置保持原样，端到端可复现；代价是多一个需要说明的配置文件，而文档还没补上它的说明。

- **决策 5：CSV 读取全量进内存，写入分批提交。**
  - 当时的依据：上传上限 10 MB 由 MAX_UPLOAD_FILE_SIZE_MB 兜住，内存占用有上界；csv.reader 在整段文本上解析最简单，也便于把「文件行号」算准（含表头、空行不计数）。写入按 200 行一批，落在规格建议的 100–500 区间。
  - 实际收益与代价：行号语义清晰、错误定位准确；代价是没有做流式读取，内存占用随文件线性增长（但被封顶在 10 MB）。

- **决策 6：金额全程用 Decimal，amount 校验拒绝科学计数法与超范围。**
  - 当时的依据：规格与 PRD 都要求金额精确、禁用浮点，且 DECIMAL(12,2) 有范围上限。
  - 实际收益与代价：19 / 19.9 / 19.90 统一归一为两位小数且不丢精度；代价是校验分支变多（负数、小数位、科学计数法、超范围、格式错各一条消息）。

- **决策 7：不实现登录与权限，也不实现取消、重试、导出。**
  - 当时的依据：PRD 明确本项目不实现登录与权限；取消、重试、导出属于 V1.1 增强方向。
  - 实际收益与代价：V1.0 范围收敛，3 周内可交付；代价是第 1 周 API 设计里列出的 /cancel、/export 与 job_attempts 表落空，设计文档需要标注取代关系。

7. 后续行动

行动	负责人	截止时间	验收标准
修正测试报告「Worker」一节的核对命令，去掉保留字 row_number，并替换含报错的 Worker日志.png	仓库所有者	V1.0 文档收尾	照抄报告命令能一次执行成功，截图无 ERROR 1064
把 docker-compose.e2e.yml 的用途写进 README 6.6 与 Release Notes「使用注意」	仓库所有者	V1.0 文档收尾	8000 不可绑时，读者能从文档找到 Compose 的可用命令
提交测试报告、Release Notes、复盘与 docs/week-03/screenshots/	仓库所有者	V1.0 文档收尾	干净 clone 后报告内 10 张图全部可显示
决定 v1.0.0 标签策略：重打标签、改发 v1.0.1，或保留标签并说明差异	仓库所有者	V1.0 收尾	标签与发布包内容的关系在文档中明确且不再矛盾
在发布 PR 上触发远端 AI Code Review（/review）	仓库所有者	V1.0 收尾	PR 上出现 Review 评论；第 3 周发布标准第 6 条由「未验证」转「已完成」
给第 1 周的 5 份设计文档加「已被第 3 周规格取代」标注，或按实现回填	仓库所有者	V1.1 启动前	设计文档与实现的差异不再需要读者自行发现
处理 frontend/src/api/errorCodes.ts：接入页面做兜底，或删除	仓库所有者	V1.1	错误码不再存在未被引用且与后端重复的定义

行动	负责人	截止时间	验收标准
为前端补关键交互测试（创建、列表、详情三态与轮询停止）	仓库所有者	V1.1	出现 `npm test` 且覆盖三个页面关键交互；PRD 9.2 达成
接入 `WORKER_CONCURRENCY`，并补 1,000 行并发基线用例	仓库所有者	V1.1	Worker 可并发消费；并发场景下无重复入库、服务不崩
评估超时回收改为周期执行，并明确回收后已写入记录与计数的处置	仓库所有者	V1.1	卡住的 `RUNNING` 可被自动回收，且统计与 `sync_records` 行数一致
开启 V1.1 议题：重试策略与执行记录、幂等键与文件摘要、任务取消、CI 自动检查、结果导出	仓库所有者	V1.1	每个议题有独立 Issue，且不在 `v1.0.0` 上改历史提交

8. 相关资料

本周产出

文档	路径	内容
测试报告	`docs/week-03/测试报告.md`	测试环境、65 个用例清单、端到端验证记录与未覆盖项
发布说明	`docs/week-03/Release-Notes-v1.0.md`	V1.0 范围、安装说明、规范口径、不包含项与文档索引
AI Code Review 记录	`docs/week-03/AI-Code-Review.md`	5 个已合并 PR 与已核对行为、已知限制
任务计划	`docs/week-03/任务计划.md`	任务拆分、依赖与里程碑、待拍板事项
演示脚本	`docs/week-03/演示脚本.md`	三分钟演示时间线与现场改规则的演示点

接口与设计

文档	路径	内容
API 文档	`docs/api.md`	已实现的 4 条接口、请求响应示例与 12 个错误码表
API 设计（第 1 周）	`docs/week-01/05-API设计.md`	接口清单与信封约定（部分路径为 V1.1 计划，见问题 4）
数据库设计	`docs/week-01/06-数据库设计.md`	表结构、类型与索引理由（表名以实现为准，见问题 4）
任务状态机	`docs/week-01/07-任务状态机.md`	状态定义与合法转换
Web 页面设计	`docs/week-01/08-Web页面设计.md`	页面、路由与三态处理
业务名词表	`docs/week-01/02-业务名词表.md`	术语口径
需求疑问清单	`docs/week-01/01-需求疑问清单.md`	第 1 周待确认与已确认记录
测试计划（第 1 周）	`docs/week-01/09-测试计划.md`	异常场景、边界值与期望结果
系统边界图	`docs/week-01/03-系统边界图.png`	系统边界示意
核心流程图	`docs/week-01/04-核心流程图.png`	CSV 同步主链路

运行与验证

文件	路径	内容
部署说明	`README.md`	技术栈、启动、停止、迁移、前端构建、测试、日志与已知限制
样例与期望统计	`samples/README.md`	5 份样例的终态与计数期望
Compose 主配置	`docker-compose.yml`	五服务编排
Compose 端到端覆盖	`docker-compose.e2e.yml`	宿主端口 8000 不可用时的 API 端口覆盖（见问题 1）
AI Review 工作流	`.github/workflows/deepseek-review.yml`	PR 上由所有者评论 `/review` 触发的远端评审
端到端截图	`docs/week-03/screenshots/`	链路搭建、探活、成功/失败任务、Worker 日志、页面检查共 10 张

需求来源

文档	路径
产品需求说明书（PRD）	`docs/PRD/product-requirements.pdf`
第 1 周任务书	`docs/tasks/week-01.pdf`
第 2 周任务书	`docs/tasks/week-02.pdf`
第 3 周任务书	`docs/tasks/week-03.pdf`

往期记录

文档	路径
第 2 周报告	`docs/week-02/week02-report.md`
第 2 周任务清单	`docs/week-02/任务清单.md`